

MetaMask Money Account

Technical & Protocol Guide

Architecture, on-chain protocol (Veda BoringVault), EIP-7702 / CHOMP upgrade, deposit & withdraw flows, balances/APY, Card linkage, and mobile implementation.

Sources: metamask-mobile (app/components/UI/Money, Engine controllers), @metamask/money-account-* packages, Veda docs (docs.veda.tech). Generated for engineering reference.

Table of contents

- 1. Executive summary
- 2. Product concept
- 3. High-level architecture
- 4. Protocol layer - Veda BoringVault
- 5. Tokens - mUSD and vmUSD
- 6. Deposit flow (protocol + app)
- 7. Withdraw flow (protocol + app)
- 8. Yield, exchange rate, and APY
- 9. EIP-7702 smart-account upgrade & CHOMP
- 10. MetaMask Card linkage
- 11. Engine packages & state
- 12. Mobile UX surfaces & feature flags
- 13. Security & risk model
- 14. Glossary & key file paths

1. Executive summary

A Money Account is a dedicated, non-exportable account address created for the user's primary HD keyring. It is MetaMask's cash + yield surface: users fund it with mUSD (or convert/fiat via MM Pay), deposits enter a Veda BoringVault on Monad and receive vmUSD vault shares that earn variable APY, and the balance can optionally delegate spend to MetaMask Card.

At the protocol layer, Money Account is not a proprietary yield engine. Yield is generated by third-party DeFi strategies (e.g. Aave, Morpho) curated inside a Veda vault, with risk oversight described in product copy as Steakhouse Financial monitoring. MetaMask owns the wallet UX, account lifecycle, transaction batching, and Card integration.

Money Account today is Monad-scoped for the vault path (chain id decimal 143 / hex 0x8f). mUSD also exists on other chains (Ethereum, Linea, BSC) for conversion/source liquidity, but the vault deposit target is Monad mUSD.

2. Product concept

What users get

- A separate Money address (not their everyday selected account).
- USD-pegged balance displayed as Money balance (mUSD liquid + vmUSD valued in mUSD).
- Variable APY on vaulted funds - no manual claim/stake UX.
- Deposit via convert crypto, move existing mUSD, or fiat (MM Pay) when region-enabled.
- Withdraw back to a selected EVM account.
- Optional Card link so purchases pull from Money while funds keep earning until spent.

What it is not

- Not interest paid by MetaMask or by the mUSD issuer.
- Not a custodial bank account - assets sit in self-custodial contracts/wallet.
- Not guaranteed APY - rates are variable and market-dependent.

3. High-level architecture

Three Engine packages plus mobile UI:

- @metamask/money-account-controller - creates/tracks Money accounts in keyring state (moneyAccounts map; exportable: false).
- @metamask/money-account-balance-service - on-chain Multicall balances, Accountant rate, Lens valuation, Veda REST APY.
- @metamask/money-account-upgrade-controller - EIP-7702 + CHOMP upgrade sequence for auto deposit/withdraw intents.

End-to-end fund path:

```

User EOA / fiat
  | MM Pay (bridge/swap/fiat into Money Account as mUSD on Monad)
  v
Money Account (EOA upgraded via EIP-7702)
  | ERC-20 approve(boringVault) + Teller.deposit(mUSD)
  v
Veda BoringVault --> user holds vmUSD shares
  | Curator/Manager deploys into DeFi (Aave, Morpho, ...)
  v
Yield increases Accountant exchange rate (vmUSD worth more mUSD)
  
```

Withdraw:

Teller.withdraw(vmUSD -> mUSD to Money Account) + ERC-20 transfer to user EOA

4. Protocol layer - Veda BoringVault

Money Account uses Veda's BoringVault architecture. The vault core is intentionally minimal (~100 lines of critical custody logic). Complex behavior is delegated to external modules that can be audited and constrained independently.

Primary reference: <https://docs.veda.tech/architecture-and-flow-of-funds>

4.1 BoringVault

Central custody contract for vault assets. Holds underlying tokens, mints/burns share tokens (vmUSD for Money), and executes management calls when authorized modules request them. Rules for deposits, withdrawals, and strategy execution live in surrounding modules.

4.2 Teller (user entry/exit)

User-facing deposit and withdraw router. On deposit it reads the Accountant exchange rate, mints the correct number of shares, and pulls assets into the BoringVault. On withdraw it burns shares and returns underlying assets.

- Share lock periods after deposit mitigate MEV/arbitrage (shares may not be transferable or redeemable during the lock).
- Variants exist (e.g. TellerWithBuffer for instant liquidity, TellerWithYieldStreaming).

MetaMask mobile encodes these Teller methods:

```
deposit(address depositAsset, uint256 depositAmount,  
        uint256 minimumMint, address referralAddress) payable returns (uint256 shares)
```

```
withdraw(address withdrawAsset, uint256 shareAmount,  
        uint256 minimumAssets, address to) returns (uint256 assetsOut)
```

4.3 Accountant (exchange rate)

Publishes the share <-> underlying exchange rate. Rates are typically computed offchain and pushed onchain with safety rails:

- Rate limiting - how often the rate can update.
- Bound limiting - how far a new rate may deviate from the previous on-chain rate.
- If bounds are violated, the Accountant can pause updates (and thus deposits/withdraws).

Mobile reads the rate via:

```
getRate() view returns (uint256 rate)
```

4.4 Lens (preview / valuation helpers)

Lens contracts provide view helpers for UIs and integrators. Money Account uses Lens for:

- previewDeposit(...) - expected shares for a deposit amount (sets minimumMint).
- balanceOfInAssets(...) - value vault shares in underlying asset terms (used by balance service as mUSD-equivalent of vmUSD).

```
previewDeposit(address depositAsset, uint256 depositAmount,  
              address boringVault, address accountant) view returns (uint256 shares)
```

4.5 Manager (strategy whitelist)

Controls which DeFi actions the vault may execute. Permitted operations are encoded in a Merkle tree (target address, function, allowed params). A strategist/curator submits rebalances with Merkle proofs; the Manager verifies them onchain. This keeps strategy policy enforceable at execution time.

4.6 DecoderAndSanitizer

Decodes and validates calldata before external protocol calls, ensuring only intended interactions (per integrated protocol) execute.

4.7 Hook

Optional pre-transfer logic: transfer restrictions, whitelists, share locks, compliance.

4.8 BoringQueue (async withdrawals)

For vaults without sufficient instant buffer liquidity, withdrawals can use a time-delayed, solver-based queue: shares move to the queue, mature, then solvers fulfill by providing assets. Money Account's mobile path currently targets Teller.withdraw (instant-style path when liquidity allows); queue behavior depends on vault configuration.

4.9 Flow of funds inside the vault

1. User deposits mUSD via Teller -> shares (vmUSD) minted at Accountant rate
2. Idle assets sit in vault / buffer
3. Curator rebalances via Manager (Merkle-verified) into DeFi markets
4. Yield accrues; Accountant rate rises over time
5. User withdraws: shares burned, mUSD returned (subject to liquidity/locks)

4.10 Vault config in MetaMask (remote feature flag)

Addresses are not hardcoded for production. Remote flag moneyAccountVaultConfig supplies:

```
MoneyAccountVaultConfig / VaultConfig =  
{  
  chainId,           // e.g. 0x8f (Monad)  
  boringVault,       // vault + share token (vmUSD)  
  tellerAddress,     // deposit / withdraw entry  
  accountantAddress, // getRate()  
  lensAddress,       // previewDeposit / valuation  
  underlyingToken?,  // optional mUSD; else Accountant.base()  
}
```

Dev fallback exists when MM_MONEY_DEPOSIT_CONFIG_DEV_ENABLED=true (DEV_VAULT_CONFIG in moneyAccount feature-flag selectors).

5. Tokens - mUSD and vmUSD

mUSD (MetaMask USD)

- USD-pegged ERC-20 used as the Money Account cash unit.
- Decimals: 6 (same pattern as USDC-style stables).
- Canonical address used across supported chains (checksum example):
0xacA92E438df0B2401fF60dA7E4337B687a2435DA
- Money Account vault path: Monad only (eip155:143). Other chains hold source mUSD for conversion / MM Pay funding.
- Display symbol in product: mUSD.

vmUSD (vault shares)

- Receipt / share token of the BoringVault (boringVault address).
- Represents claim on vault assets + accrued yield via rising exchange rate.
- UI usually shows total Money balance in fiat/mUSD terms, not raw share count.

Balance composition (app)

```
MoneyAccountBalanceResponse =
{
  musdBalance,          // liquid mUSD on Money Account
  vmusdValueInMUSD,     // vault shares valued in mUSD (Lens)
  totalBalance          // musdBalance + vmusdValueInMUSD (raw uint256 strings)
}
```

6. Deposit flow (protocol + app)

6.1 Protocol steps

- Ensure Money Account holds mUSD on the vault chain (via MM Pay).
- Approve: mUSD.approve(boringVault, amount) - spender is the vault (as encoded in app).
- Preview: Lens.previewDeposit(mUSD, amount, boringVault, accountant) -> expected shares.
- Apply 0.2% slippage: $\text{minimumMint} = \text{expectedShares} * 998 / 1000$.
- Teller.deposit(mUSD, amount, minimumMint, referral=0x0) -> mints vmUSD to depositor (Money Account).

6.2 App orchestration

- Hook: useMoneyAccountDeposit (app/components/UI/Money/hooks/useMoneyAccount.ts).
- Builder: buildMoneyAccountDepositBatch (moneyAccountTransactions.ts).
- Submitted as TransactionController atomic batch from = Money Account address.
- MM Pay selects payment token / fiat and re-encodes amounts via updateMoneyAccountDepositTokenAmount when the user sets amount.
- On Monad mainnet, gas is typically sponsored (isGasFeeSponsored).
- Entry sheet MoneyAddMoneySheet offers convert crypto / fiat / move mUSD (plus coming-soon bank & receive rows).

6.3 Slippage rationale

minimumMint protects against rate movement between preview and inclusion. If the sanity check would revert, funds are not lost - user retries with a fresh quote. Constant: SLIPPAGE_NUMERATOR=998, SLIPPAGE_DENOMINATOR=1000.

7. Withdraw flow (protocol + app)

7.1 Protocol steps

- Read Accountant.getRate().
- Convert desired mUSD amount to shares with ceiling division so mulDivDown on-chain does not undershoot minimumAssets (avoids MinimumAssetsNotMet).

```
SHARE_DECIMALS_SCALAR = 1_000_000
shares = ceil(amount * SHARE_DECIMALS_SCALAR / rate)
minimumAssets = amount - 1 # 1-unit defense-in-depth vs rounding
```

- Teller.withdraw(mUSD, shareAmount, minimumAssets, to=MoneyAccount) - underlying lands on Money Account first.
- ERC-20 transfer(recipient, amount) moves exact requested mUSD to the user's selected EOA.

7.2 App orchestration

- Hook: useMoneyAccountWithdrawal.
- Builder: buildMoneyAccountWithdrawBatch.
- Amount updates: updateMoneyAccountWithdrawTokenAmount.

Comments in code sometimes say USDC; operational asset for Money Account is mUSD.

8. Yield, exchange rate, and APY

Yield is not streamed as a separate claim token in the Money UX. As strategies earn, the Accountant rate increases so each vmUSD share is redeemable for more mUSD. The app displays APY from Veda's REST API (MoneyAccountBalanceService.getVaultApy), with optional remote override/fallback flags (earnMoneyVaultApyControl).

NormalizedVaultApyResponse includes apy (decimal), timestamp, optional fees and breakdowns.

Product messaging: APY is variable and not guaranteed; earnings come from onchain returns in third-party protocols, not from MetaMask or the mUSD issuer.

9. EIP-7702 smart-account upgrade & CHOMP

Money Account starts as a keyring account. To enable automated vault interactions (auto deposit/withdraw), MoneyAccountUpgradeController runs an idempotent upgrade sequence integrating MetaMask Delegation Framework + CHOMP.

9.1 Bootstrap

- Requires moneyEnableMoneyAccount + moneyAccountVaultConfig.
- Ensures vault chainId exists in NetworkController (auto-adds Monad from PopularList).
- controller.init({ chainId, boringVaultAddress }).
- UI awaits whenMoneyAccountUpgradeReady() then upgradeAccount(address).

9.2 Upgrade steps (ordered, skip if already-done)

- 1. eip-7702-authorization - Sign EIP-7702 authorization; submit to CHOMP (409 treated as already authorized). Sets smart-account delegation slot.
- 2. associate-address - Sign personal message 'CHOMP Authentication {timestamp}' (EIP-191); CHOMP links address to user profile.
- 3. build-delegations - Build, sign, CHOMP-verify, and persist ERC-20 delegations for auto-deposit (mUSD) and auto-withdraw (vmUSD / boring vault). Caveats include ValueLteEnforcer, ERC20TransferAmountEnforcer (max uint256), RedeemerEnforcer (Veda vault adapter only).
- 4. register-intents - Register CHOMP intents for those delegations so CHOMP can monitor and execute auto deposit/withdraw flows.

9.3 Delegation shape (simplified)

```
delegation = {
  delegate: CHOMP_delegate,
  delegator: moneyAccountAddress,
  authority: ROOT_AUTHORITY,
  caveats: [valueLte, erc20TransferAmount, redeemer],
  salt: random(32),
  signature: DelegationController.signDelegation(...)
}
```

Stored via AuthenticatedUserStorageService with metadata (delegationHash, chainId, token, type). CHOMP rejects invalid delegations before persistence.

10. MetaMask Card linkage

Card spend can be linked to Money via spending-limit style delegation on the vault/veda token configuration (Card APIs key it as 'veda'; UI shows mUSD).

- Hook: useMoneyAccountCardLinkage.

- Gates: Money enabled, geo eligible, vault config, Money address, Card auth/verified, etc.
- `CardController.linkMoneyAccountCard({ moneyAccountAddress, delegationAmountHuman })`.
- Unlink submits `approve(0)` style revoke and marks not-delegated on Card backend (see developer options / Card settings copy).

11. Engine packages & state

MoneyAccountController

```
state = { moneyAccounts: { [id]: MoneyAccount } }
```

- MoneyAccount is a KeyringAccount with options.entropy and exportable: false.
- init() creates account for primary entropy when flag on + unlocked.
- clearState() when flag turns off.
- selectPrimaryMoneyAccount matches options.entropy.id to primary HD keyring.

MoneyAccountBalanceService

- getMusdBalance / getVmUSDBalance / getMoneyAccountBalance
- getExchangeRate (Accountant)
- getMusdEquivalentValue (Lens)
- getVaultApy (Veda REST)
- Reads vault config from remote flags; validates config (VaultConfigValidationError).

MoneyAccountUpgradeController

- Stateless (Record<string, never>).
- Side-effect controller: init + upgradeAccount.

12. Mobile UX surfaces & feature flags

Primary UI

- app/components/UI/Money/ - Home, Onboarding (Rive), Activity, How it works, First-time deposit.
- Sheets: Add Money, Transfer, Geo block, Card link, APY info.
- Confirmations: money-account-deposit / withdraw info + MM Pay amount editing.

Important flags

- moneyEnableMoneyAccount - master version-gated kill switch.
- moneyAccountVaultConfig - protocol addresses + chainId.
- earnMoneyHubEnabled - hub discovery.
- moneyAccount.moneyAccountDepositEnabled / WithdrawEnabled.
- moneyAccountGeoBlockedCountries (default includes GB).
- earnMoneyVaultApyControl - APY override/fallback.
- Various UX polish flags (parallax, first-time deposit animation, etc.).

Geo eligibility

selectIsMoneyAccountGeoEligible fails closed when geolocation is unknown (null) for regulatory compliance - eligible users may briefly look blocked while Ramps geo loads.

13. Security & risk model

- Self-custodial: user keys control Money Account; vault shares are onchain receipts.
- Vault strategy risk: smart contract / market / curator risk in DeFi venues.
- Accountant pause / bound limits reduce oracle manipulation blast radius.
- Teller share locks reduce deposit-withdraw arbitrage.
- Manager Merkle whitelist constrains strategist actions.

- CHOMP delegations constrained by redeemer + amount enforcers.
- Money Account keyring accounts are non-exportable.
- Slippage / minimumMint / minimumAssets protect users from stale quotes.

This document does not replace security audits of Veda contracts, CHOMP, or MetaMask controllers.

14. Glossary & key file paths

Glossary

BoringVault: Minimal Veda vault custody + share mint/burn contract.

Teller: User deposit/withdraw router for the vault.

Accountant: On-chain publisher of share exchange rate.

Lens: View helpers: previewDeposit, balance valuation.

Manager: Merkle-verified strategy execution gate.

vmUSD: Vault share token for Money Account.

mUSD: MetaMask USD stablecoin; deposit asset.

CHOMP: Service that coordinates EIP-7702 upgrade + auto intents.

EIP-7702: EOA code-delegation upgrade enabling smart-account behavior.

MM Pay: Confirmation payment layer that funds Money Account then runs vault txs.

Delegation: Signed caveated authorization for limited token actions.

Key paths (metamask-mobile)

- app/core/Engine/Engine.ts - controller registration
- app/core/Engine/controllers/money-account-*.init.ts
- app/selectors/moneyAccountController/
- app/selectors/featureFlagController/moneyAccount/
- app/lib/Money/feature-flags.ts
- app/actions/money/index.ts - upgradeMoneyAccount thunk
- app/components/UI/Money/hooks/useMoneyAccount.ts
- app/components/UI/Money/hooks/useMoneyAccountBalance.ts
- app/components/UI/Money/utils/moneyAccountTransactions.ts
- app/components/UI/Card/hooks/useMoneyAccountCardLinkage.tsx
- app/components/UI/Earn/constants/musd.ts
- node_modules/@metamask/money-account-{controller,balance-service,upgrade-controller}/

External docs

- Veda architecture: <https://docs.veda.tech/architecture-and-flow-of-funds>
- Veda core components: <https://docs.veda.tech/architecture-and-flow-of-funds/core-components>
- BoringVault repo: <https://github.com/Veda-Labs/boring-vault>